

APPLICAZIONE MATCHUP

CORSO DI INGEGNERIA DEL SOFTWARE A.A. 2025 / 2026

DOCUMENTO DI PROGETTO ITERAZIONE 2

SALVATORE CAMPIONE

MARIO CIPRIANO

Sommario

1.REQUISITI.....	4
2. ANALISI	5
2.1 MODELLO DI DOMINIO	5
2.2 DIAGRAMMA DI SEQUENZA DI SISTEMA DI UC1.....	6
2.3 DIAGRAMMA DI SEQUENZA DI SISTEMA DI UC2.....	7
2.4 PERFEZIONAMENTO DIAGRAMMA DI SEQUENZA DI UC3	7
2.5 CONTRATTI DELLE OPERAZIONI UC1	8
2.5.1 ISTANZIA NUOVO GESTORE.....	8
2.5.2 INSERISCI CREDENZIALI	8
2.5.3 CONFERMA REGISTRAZIONE GESTORE	8
2.5.4 INSERISCI DATI STRUTTURA	9
2.5.5 INSERISCI CAMPO STRUTTURA.....	9
2.5.6 CONFERMA REGISTRAZIONE STRUTTURA.....	9
2.6 CONTRATTI DELLE OPERAZIONI UC2	10
2.6.1 ISTANZIA NUOVO GIOCATORE	10

2.6.2 INSERISCI CREDENZIALI	10
2.6.3 CONFERMA REGISTRAZIONE	10
2.7 PERFEZIONAMENTO DEI CONTRATTI DELLE OPERAZIONI DI UC3.....	11
2.7.1 LOGIN.....	11
2.7.2 RICHIEDI DISPONIBILITA'	11
3 PROGETTAZIONE	12
3.1 DIAGRAMMA DI SEQUENZA DI UC1	12
3.1.1 ISTANZIA NUOVO UTENTE	12
3.1.2 INSERISCI CREDENZIALI.....	12
3.1.3 CONFERMA REGISTRAZIONE UTENTE.....	14
3.1.4 INSERISCI DATI STRUTTURA	14
3.1.5 INSERISCI CAMPO STRUTTURA.....	15
3.1.6 CONFERMA REGISTRAZIONE STRUTTURA.....	15
3.2 DIAGRAMMI DI SEQUENZA DI UC2	16
3.3 DIAGRAMMI DI SEQUENZA DI UC3 PERFEZIONATI	16
3.3.1 LOGIN.....	16
3.3.2 RICHIEDI DISPONIBILITA'	17
3.3.3 MOSTRA FASCE ORARIE.....	17
3.3.4 PRENOTA CAMPO	18
3.3.5 CONTROLLA DISPONIBILITA' CAMPO	18
3.3.6 CALCOLA COSTO TOTALE.....	19
3.3.7 CONFERMA PRENOTAZIONE.....	19
3.4 DIAGRAMMA DELLE CLASSI DI PROGETTO	20
4.PATTERN UTILIZZATI	20
5.FASE DI TESTING	22
5.1 INTRODUZIONE AL TESTING	22
5.2 SCENARIO REGISTRAZIONE GIOCATORE	22
5.3 SCENARIO REGISTRAZIONE STRUTTURA.....	23
5.4 SCENARIO PRENOTA CAMPO (UC3 PERFEZIONATO)	24

1.REQUISITI

In questa seconda iterazione l'attenzione si sposta invece verso la gestione della fase di registrazione delle strutture e degli utenti presso la piattaforma e in più verranno gestiti gli aspetti legati all'autenticazione. Questa operazione viene introdotta qui ma essa varrà anche per i successivi casi d'uso e le successive iterazioni dove è richiesta e verrà utilizzata sia per autenticare il gestore che il giocatore.

Più precisamente in questa iterazione verranno analizzati i seguenti casi d'uso:

- UC1: Registrazione Struttura
- UC2 Registrazione Giocatore
- Perfezionamento UC3 (Effettua Prenotazione)

Per quest'ultimo verrà introdotta la versione del caso d'uso che prevede l'autenticazione del Giocatore e terrà conto anche delle prenotazioni dei campi effettuate in precedenza dai giocatori, che saranno coinvolte nel calcolo del costo delle prenotazioni secondo le regole di dominio .

2. ANALISI

Come per la prima iterazione anche qui in questa sezione andremo a definire il modello di dominio , i vari diagrammi di sequenza e i contratti delle operazioni.

2.1 MODELLO DI DOMINIO

In questa iterazione è stato modificato il modello di dominio precedente, infatti dall'UC1 , è stata individuata una nuova classe progettuale chiamata "Gestore", sono stati definiti i suoi attributi e inoltre è stata arricchita la classe struttura con nuovi attributi specifici per essa (orario di apertura, orario di chiusura, numero di telefono) .

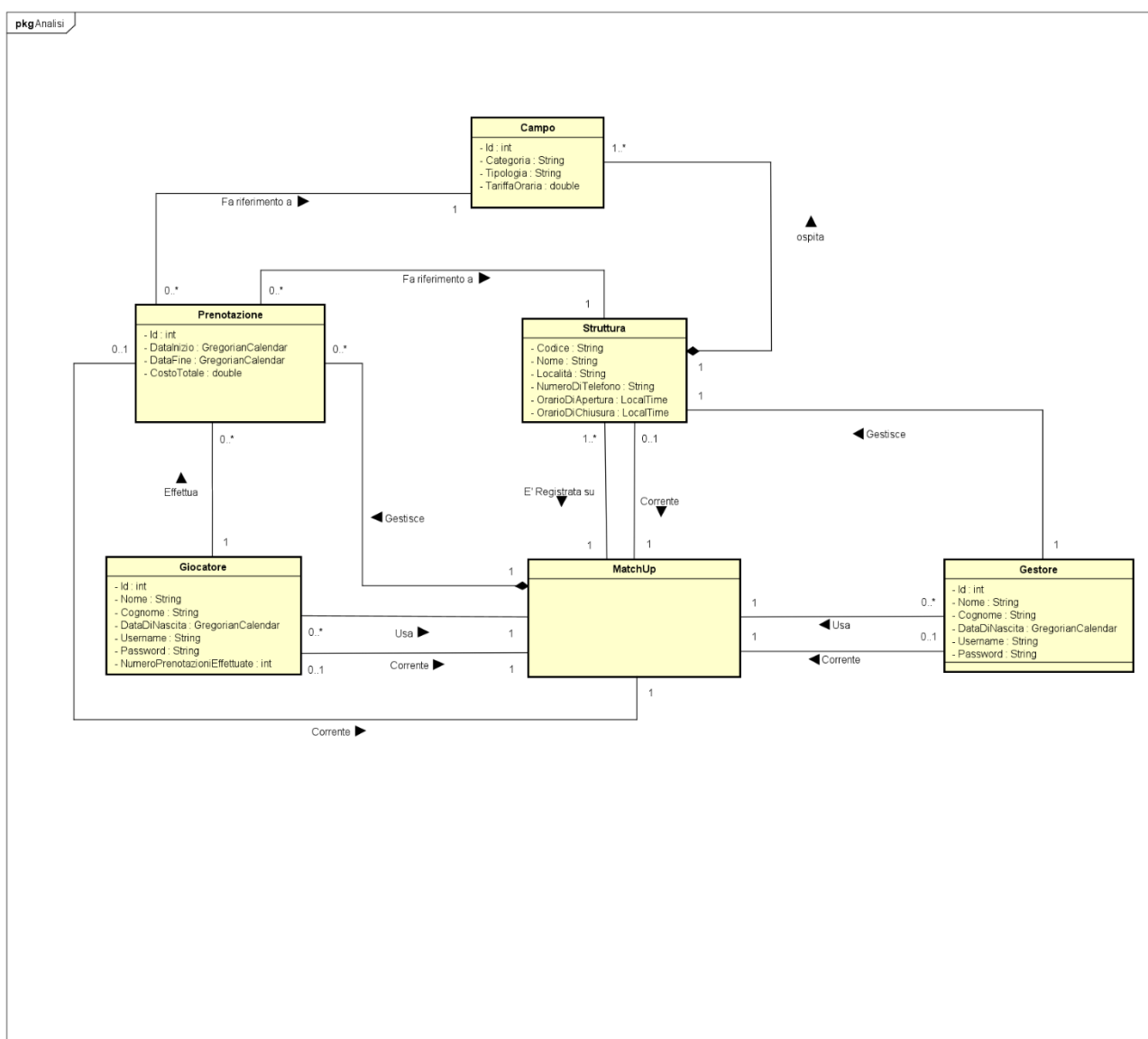


Figura 1: Modello di dominio

2.2 DIAGRAMMA DI SEQUENZA DI SISTEMA DI UC1

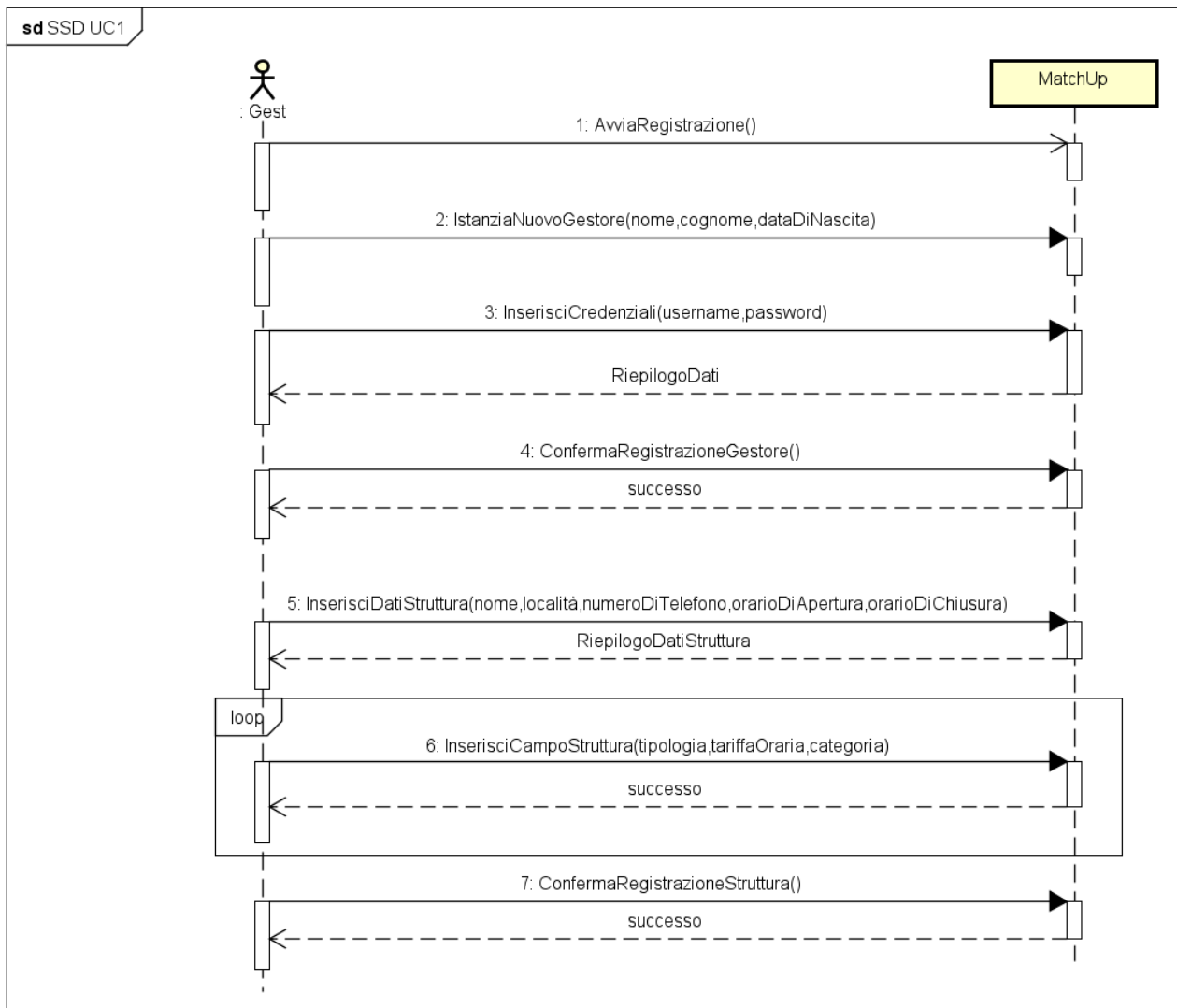


Figura 2: Diagramma di sequenza di sistema UC1

2.3 DIAGRAMMA DI SEQUENZA DI SISTEMA DI UC2

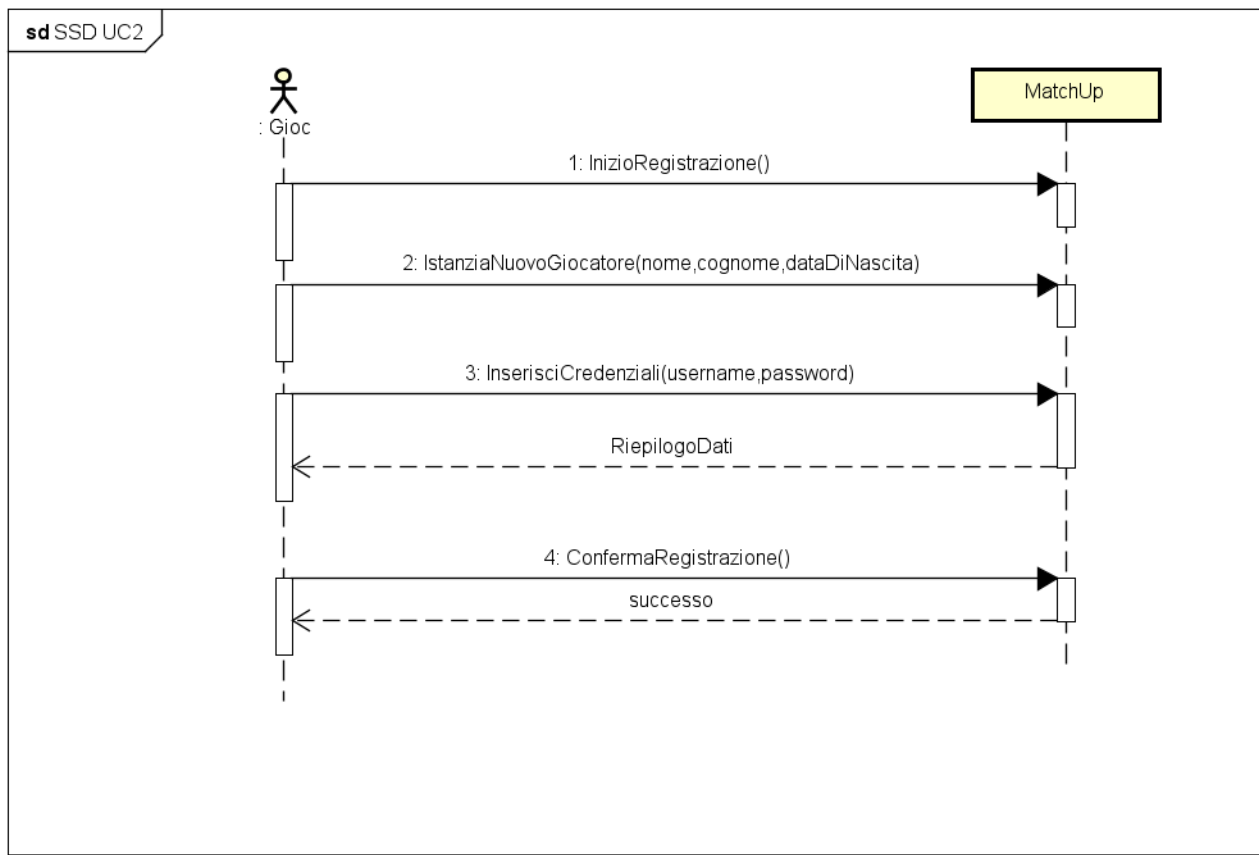


Figura 3: Diagramma di sequenza di sistema UC2

2.4 PERFEZIONAMENTO DIAGRAMMA DI SEQUENZA DI UC3

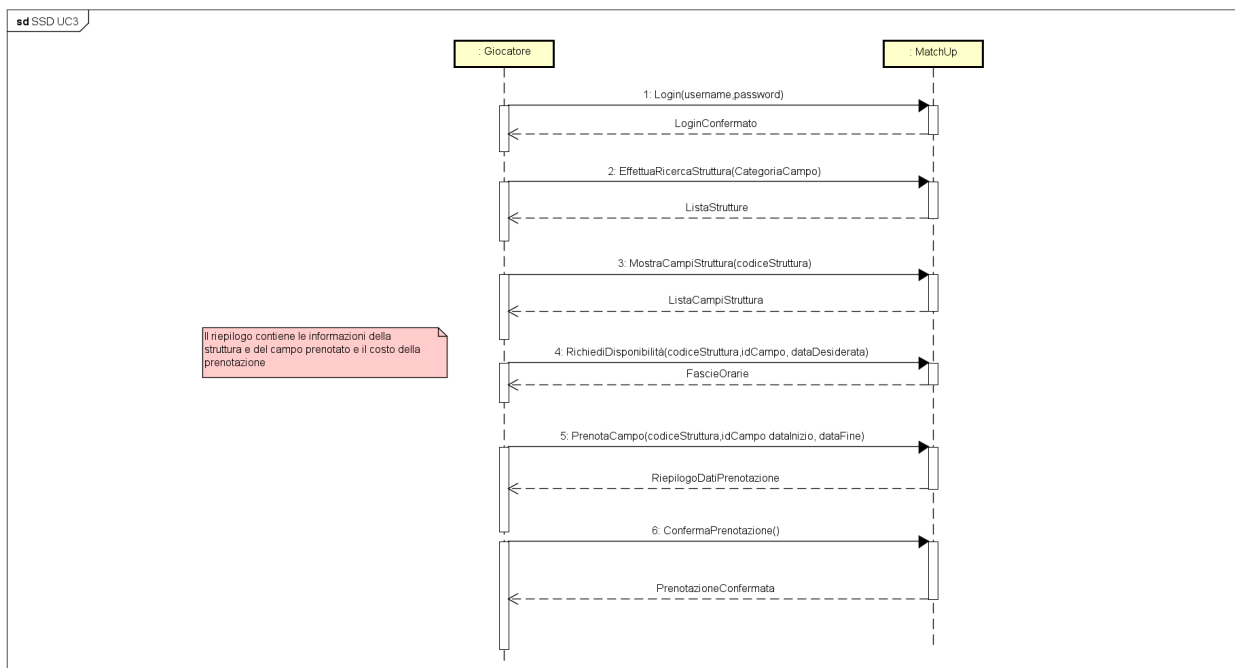


Figura 4: Diagramma di sequenza di sistema UC3 perfezionato

2.5 CONTRATTI DELLE OPERAZIONI UC1

2.5.1 ISTANZIA NUOVO GESTORE

Questa operazione permette al sistema di creare un'istanza di Gestore ge con i dati che quest'ultimo ha inserito.

Operazione	IstanziaNuovoGestore(nome:String,cognome:String,dataDiNascita:GregorianCalendar)
Riferimenti	Caso d'uso UC1 (Registrazione Struttura)
Pre-condizioni	-
Post-condizioni	- E' stata creata un'istanza di Gestore ge - Gli attributi dell'istanza sono stati inizializzati (nome,cognome,dataDiNascita) - L'istanza ge è stata associata a MatchUp tramite l'associazione "corrente"

Tabella 1: Contratto dell'operazione Istanza nuovo gestore

2.5.2 INSERISCI CREDENZIALI

Questa operazione permette di associare le credenziali al gestore che si sta registrando

Operazione	InserisciCredenziali(username:String,password:String)
Riferimenti	Caso d'uso UC1 (Registrazione Struttura)
Pre-condizioni	- E' in corso la registrazione di un Gestore ge
Post-condizioni	- Sono stati settati gli attributi username e password dell'istanza ge

Tabella 2: Contratto dell'operazione Inserisci credenziali

2.5.3 CONFERMA REGISTRAZIONE GESTORE

Questa operazione permette di confermare i dati inseriti dal gestore e di registrare l'istanza creata.

Operazione	ConfermaRegistrazioneGestore()
Riferimenti	Caso d'uso UC1 (Registrazione Struttura)
Pre-condizioni	- L'istanza ge è stata creata
Post-condizioni	- L'istanza ge è stata associata a MatchUp tramite l'associazione "Usa"

Tabella 3: Contratto dell'operazione Conferma registrazione gestore.

2.5.4 INSERISCI DATI STRUTTURA

Questa operazione permette al gestore di inserire i dati della struttura da registrare sul sistema.

Operazione	InserisciDatiStruttura(nome:String,località:String, numeroTelefono: String, OrarioApertura:LocalTime, OrarioChiusura: LocalTime)
Riferimenti	Caso d'uso UC1 (Registrazione Struttura)
P re-condizioni	- Il gestore è stato registrato nel sistema
Post-condizioni	- E' stata creata un'istanza di struttura s - Gli attributi dell'istanza s sono stati inizializzati - L'istanza s è stata associata a MatchUp tramite l'associazione "corrente"

Tabella 4 : Contratto dell'operazione Inserisci dati struttura

2.5.5 INSERISCI CAMPO STRUTTURA

Questa operazione permette al gestore di inserire i dati su un campo relativo ad una struttura da registrare sul sistema.

Operazione	InserisciCampoStruttura(categoriaCampo:String, tipologia:String,tariffaOraria:Double)
Riferimenti	Caso d'uso UC1 (Registrazione Struttura)
P re-condizioni	- E' in corso la registrazione di una struttura s
Post-condizioni	- E' stata creata un'istanza di campo c - Gli attributi dell'istanza c sono stati inizializzati - L'istanza c è stata associata alla struttura s tramite l'associazione "ospita"

Tabella 5 : Contratto dell'operazione Inserisci campo struttura

2.5.6 CONFERMA REGISTRAZIONE STRUTTURA

Questa operazione permette di confermare i dati e di registrare la struttura nel sistema

Operazione	ConfermaRegistrazioneStruttura()
Riferimenti	Caso d'uso UC1 (Registrazione Struttura)
Pre-condizioni	-Il gestore è stato registrato - E' in corso la registrazione di una struttura s

Post-condizioni	- L'istanza s è stata associata a MatchUp tramite l'associazione "E' registrata su" - L'istanza s è stata associata a Gestore ge tramite l'associazione "gestisce"
------------------------	---

Tabella 6 : Contratto dell'operazione Conferma registrazione struttura

2.6 CONTRATTI DELLE OPERAZIONI UC2

2.6.1 ISTANZIA NUOVO GIOCATORE

Questa operazione permette al sistema di creare un'istanza di Giocatore g con i dati che quest'ultimo ha inserito.

Operazione	IstanziaNuovoGiocatore(nome:String,cognome:String,dataDiNascita:GregorianCalendar)
Riferimenti	Caso d'uso UC2 (Registrazione Giocatore)
Pre-condizioni	-
Post-condizioni	- E' stata creata un'istanza di Giocatore g - Gli attributi dell'istanza sono stati inizializzati (nome,cognome,dataDiNascita) - L'istanza g è stata associata a MatchUp tramite l'associazione "corrente"

Tabella 7: Contratto dell'operazione Istanza nuovo giocatore

2.6.2 INSERISCI CREDENZIALI

Questa operazione permette di associare le credenziali al giocatore che si sta registrando, verificando in particolar modo che lo username sia valido e non sia già in uso da parte di un altro giocatore e che la password sia valida.

Operazione	InserisciCredenziali(username:String,password:String)
Riferimenti	Caso d'uso UC2 (Registrazione Giocatore)
Pre-condizioni	- E' in corso la registrazione di un Giocatore g
Post-condizioni	- Sono stati settati gli attributi username e password dell'istanza g

Tabella 8: Contratto dell'operazione Inserisci credenziali

2.6.3 CONFERMA REGISTRAZIONE

Questa operazione permette di confermare i dati inseriti dal giocatore e di registrare l'istanza creata.

Operazione	ConfermaRegistrazione()
Riferimenti	Caso d'uso UC2 (Registrazione Giocatore)

Pre-condizioni	- L'istanza g è stata creata
Post-condizioni	- L'istanza g è stata associata a MatchUp tramite l'associazione "Usa"

Tabella 9: Contratto dell'operazione Conferma registrazione.

2.7 PERFEZIONAMENTO DEI CONTRATTI DELLE OPERAZIONI DI UC3

In questa versione del caso dell'UC3 abbiamo introdotto l'operazione di login e soprattutto abbiamo aggiunto la possibilità di scegliere la fascia di orario disponibile in cui andare a prenotare il campo ,dopo che il giocatore ha scelto la data in cui vorrebbe effettuare la prenotazione. Inoltre abbiamo anche fatto in modo che l'oggetto Prenotazione abbia un riferimento a struttura e campo ,in modo tale che la disponibilità possa essere verificata più facilmente senza gestire le singole liste per ogni campo, come avveniva nella prima iterazione.

2.7.1 LOGIN

Questa operazione permette a chi utilizza il sistema di autenticarsi prima di usufruire dei servizi(vale sia per i giocatori che per i gestori).Infatti per semplicità useremo l'istanza "utente u" per riferirci alle istanze di entrambe le classi coinvolte.

Operazione	Login(username:String , password:String)
Riferimenti	Caso d'uso UC3 (Effettua prenotazione)
Pre-condizioni	- L'utente (giocatore o gestore) devono essere registrati nel sistema
Post-condizioni	- L'istanza dell'utente u è stata associata a MatchUp tramite l'associazione "corrente"

Tabella 10: Contratto dell'operazione Login

2.7.2 RICHIEDI DISPONIBILITA'

Questa operazione , fornita una data dal giocatore, permette di ritornare tutte le fasce orarie libere e non per prenotare il campo.

Operazione	RichiediDisponibilità(codiceStruttura:String,idCampo:String, dataDesiderata:GregorianCalendar)
Riferimenti	Caso d'uso UC3 (Effettua prenotazione)
Pre-condizioni	- E' in corso una prenotazione p - La struttura e il campo relativi alla prenotazione esistono

Post-condizioni	- Vengono ritornate al giocatore le fasce orarie in base alla data che ha inserito.
------------------------	---

Tabella 11: Contratto dell'operazione Richiedi disponibilità

3 PROGETTAZIONE

Come nella prima iterazione anche qui andiamo a definire i diagrammi di sequenza e il diagramma delle classi di progetto(DCD).

3.1 DIAGRAMMA DI SEQUENZA DI UC1

3.1.1 ISTANZIA NUOVO UTENTE

Con questa operazione generalizziamo sia il caso di istanziazione del gestore che quella del giocatore per evitare di ripetere nuovamente il grafico nell'UC2 ,in quanto il processo è lo stesso ,ma cambia solo l'entità principale che viene coinvolta.

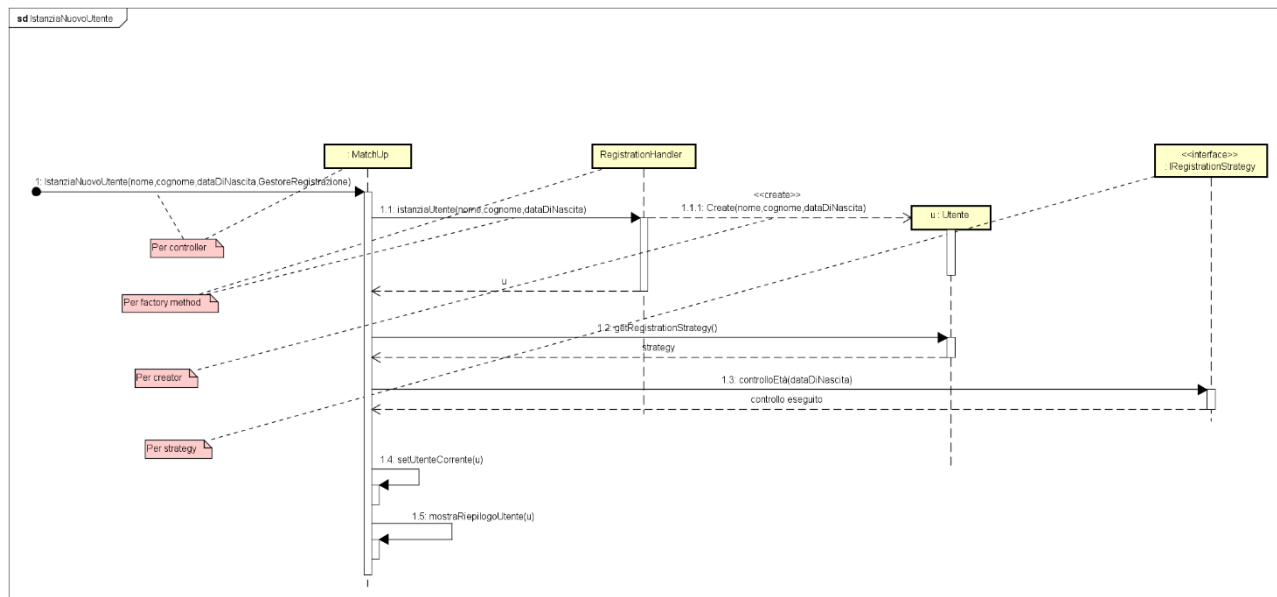


Figura 5: Diagramma di sequenza di Istanzia nuovo utente.

3.1.2 INSERISCI CREDENZIALI

Anche qui abbiamo pensato di utilizzare il diagramma solo una volta per poter rappresentare l'operazione che sarà identica anche per l'UC2 (registrazione giocatore).

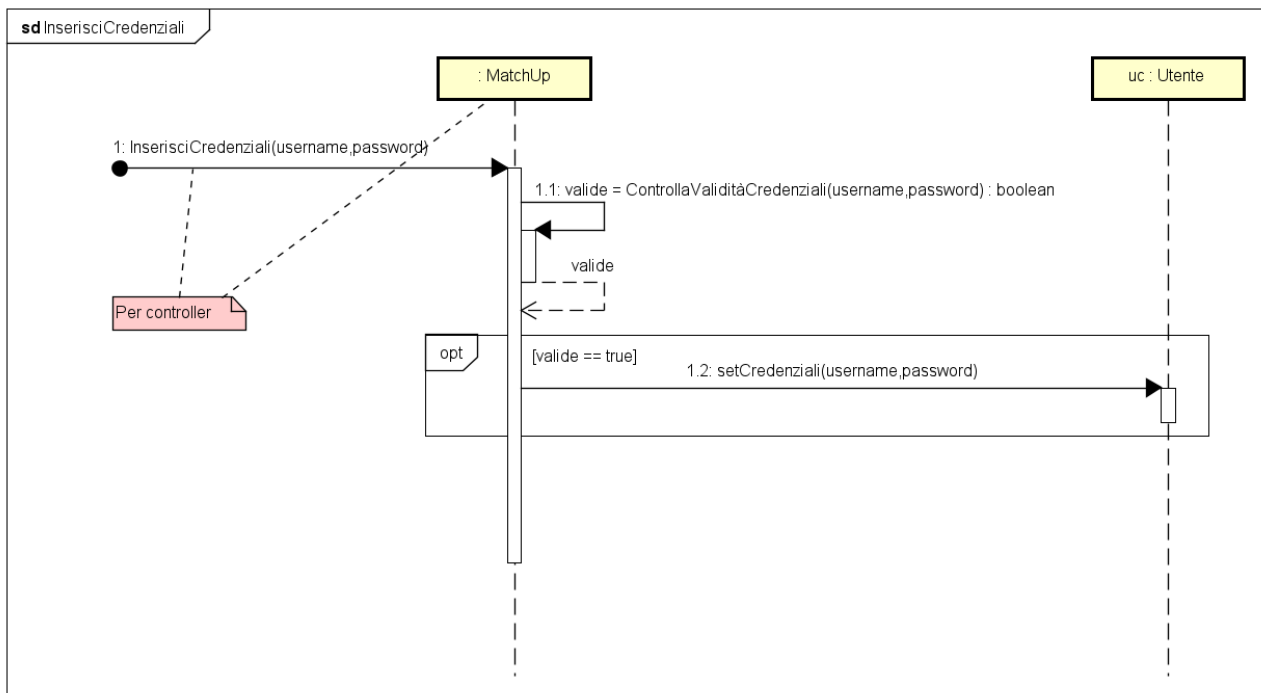


Figura 6: Diagramma di sequenza di Inserisci credenziali

3.1.3 CONFERMA REGISTRAZIONE UTENTE

Qui vale lo stesso fatto per le operazioni precedenti. Utilizzeremo una versione dell'operazione generica che sia valida sia per l'entità giocatore che gestore ,senza andare a ripetere il tutto nel UC2.

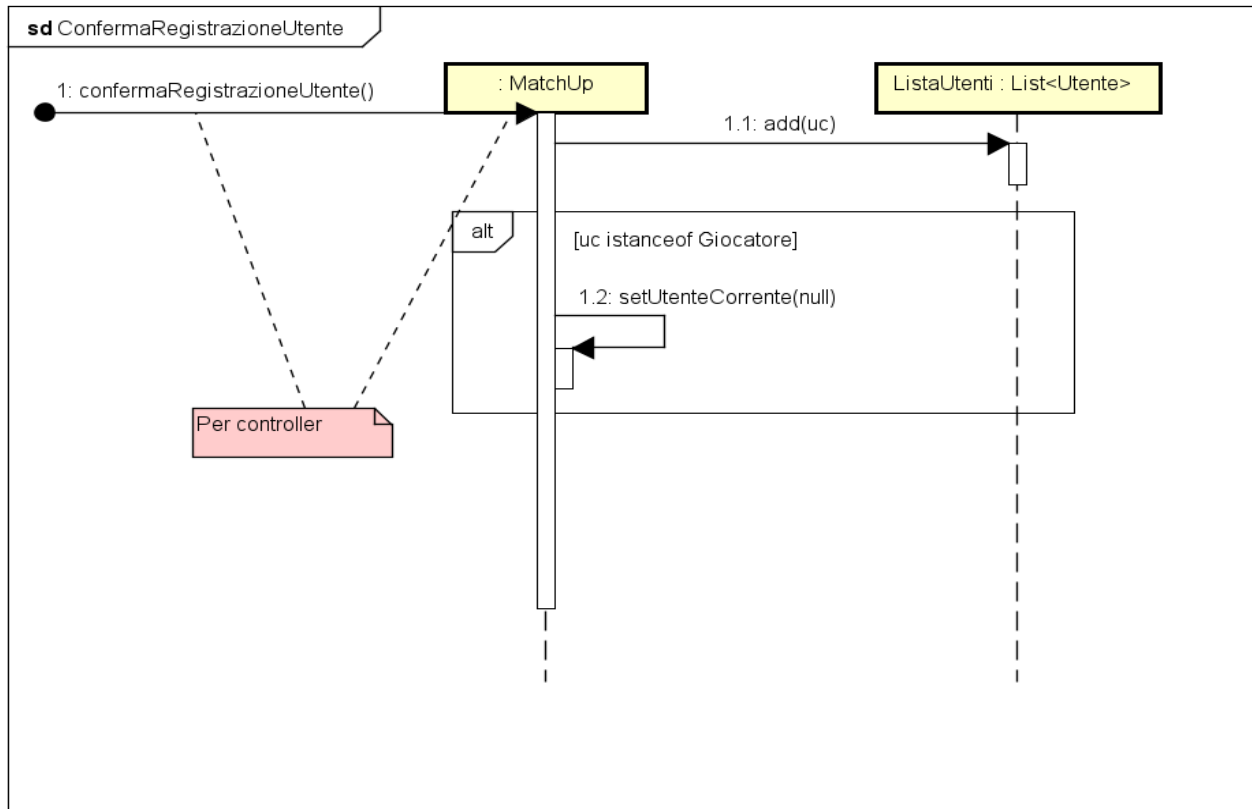


Figura 7: Diagramma di sequenza di Conferma registrazione utente

3.1.4 INSERISCI DATI STRUTTURA

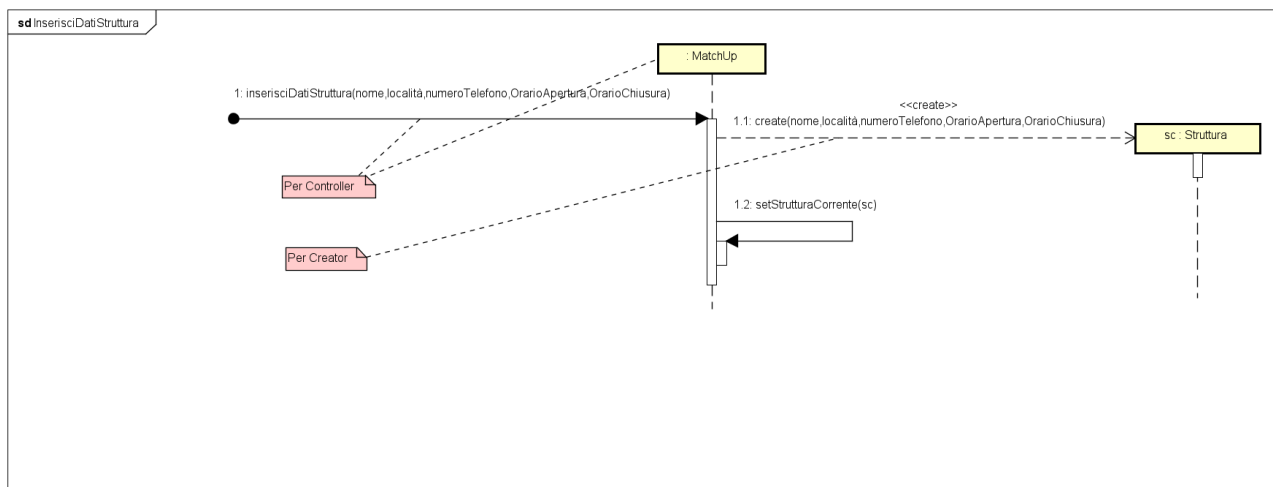


Figura 8: Diagramma di sequenza di Inserisci dati struttura

3.1.5 INSERISCI CAMPO STRUTTURA

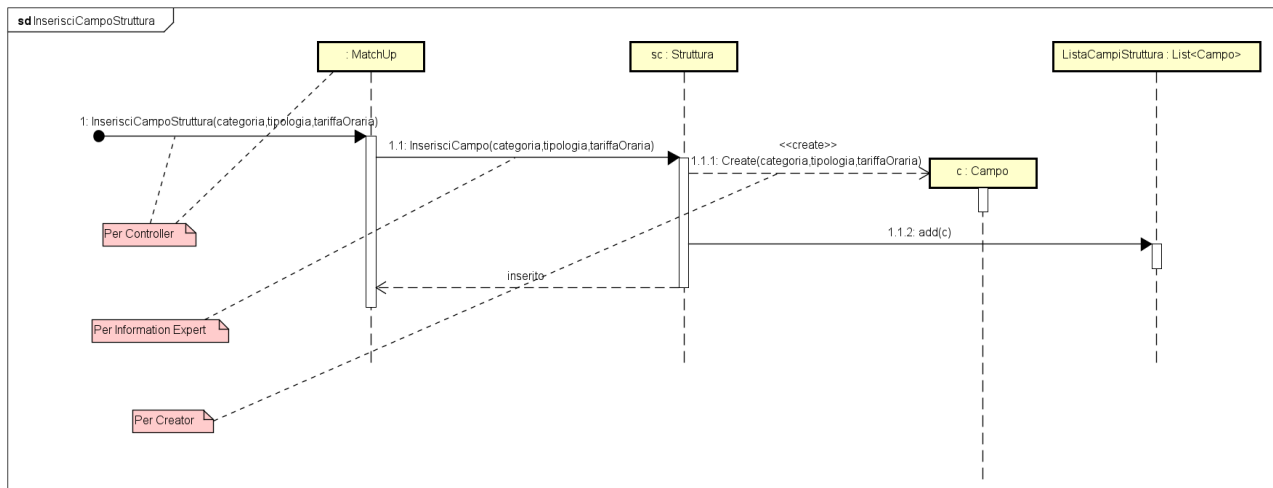


Figura 9: Diagramma di sequenza di Inserisci campo struttura

3.1.6 CONFERMA REGISTRAZIONE STRUTTURA

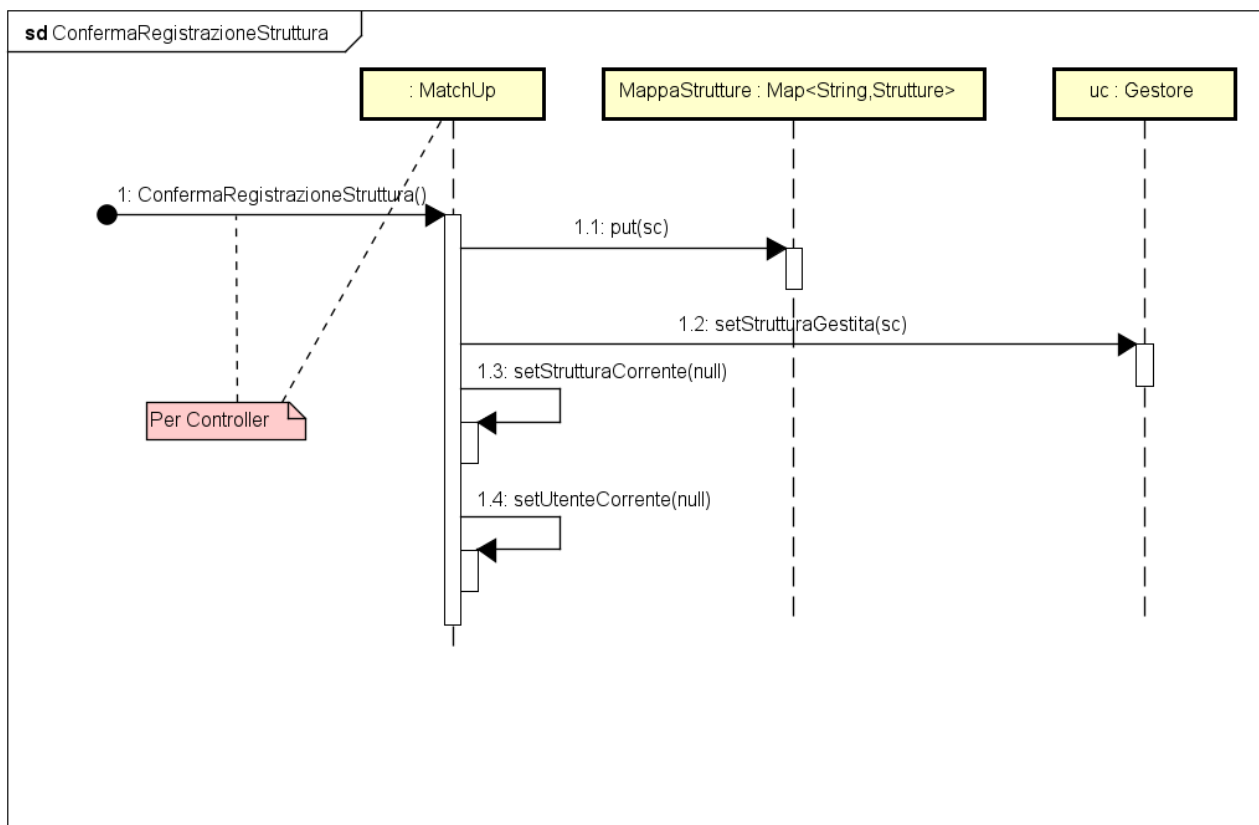


Figura 10: Diagramma di sequenza di Conferma registrazione struttura

3.2 DIAGRAMMI DI SEQUENZA DI UC2

Le operazioni eseguite nel caso d'uso UC2 corrispondono alle operazioni da 3.1.1 a 3.1.3 dell'UC1. L'oggetto di tipo utente in queste operazioni corrisponde ad un tipo Giocatore, a differenza del caso d'uso UC1 dove l'oggetto è di tipo Gestore.

3.3 DIAGRAMMI DI SQUENZA DI UC3 PERFEZIONATI

3.3.1 LOGIN

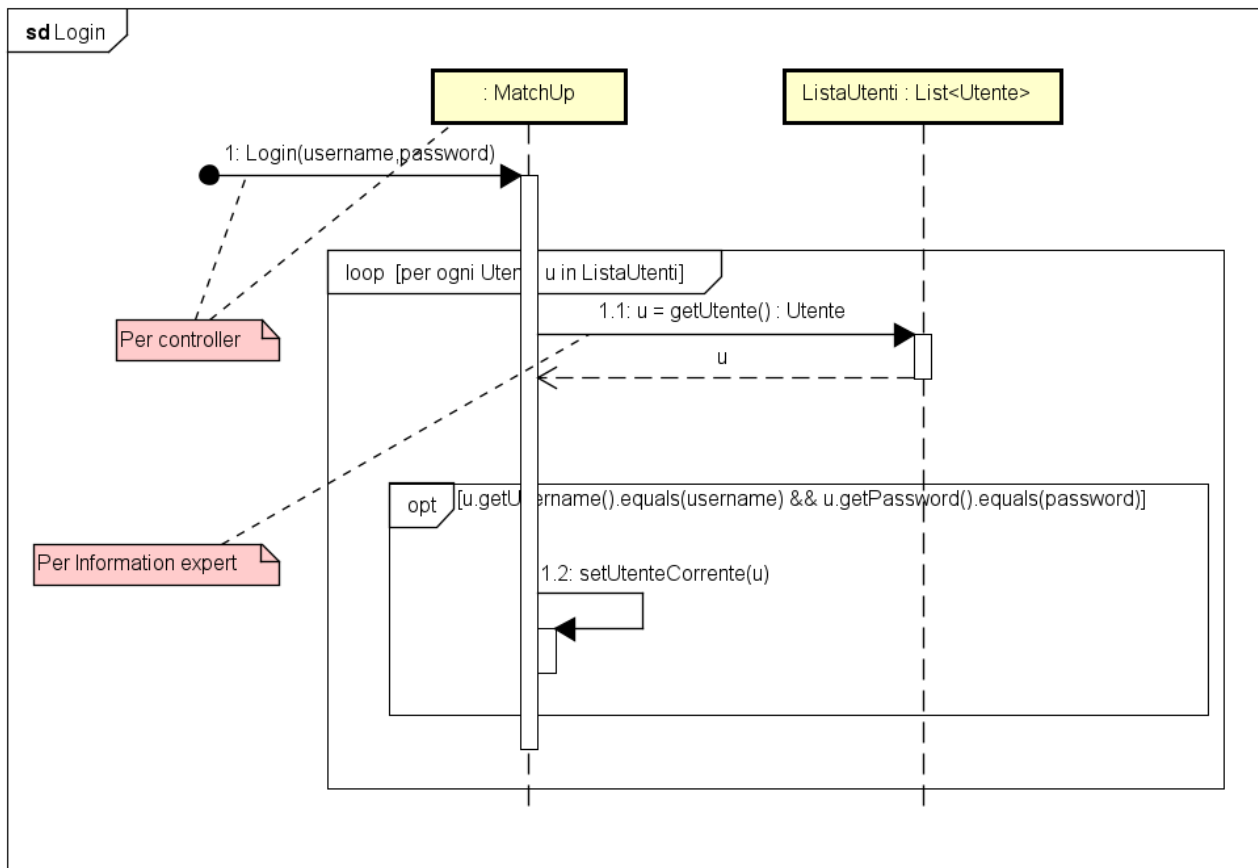


Figura 11: Diagramma di sequenza di Login

3.3.2 RICHIEDI DISPONIBILITA'

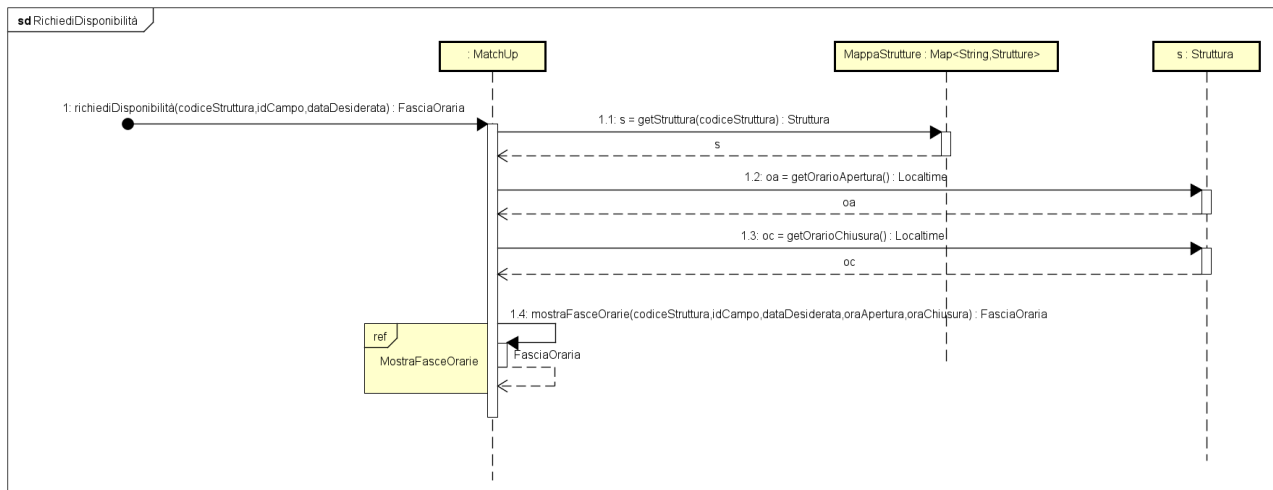


Figura 12: Diagramma di sequenza di Richiedi disponibilità

3.3.3 MOSTRA FASCE ORARIE

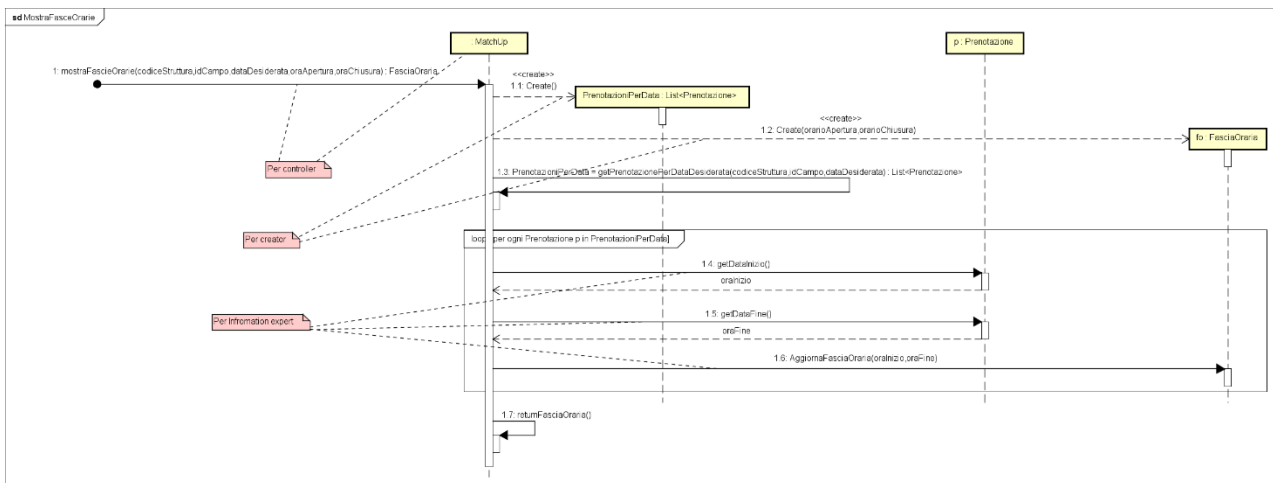


Figura 13: Diagramma di sequenza di Mostra fasce orarie

3.3.4 PRENOTA CAMPO

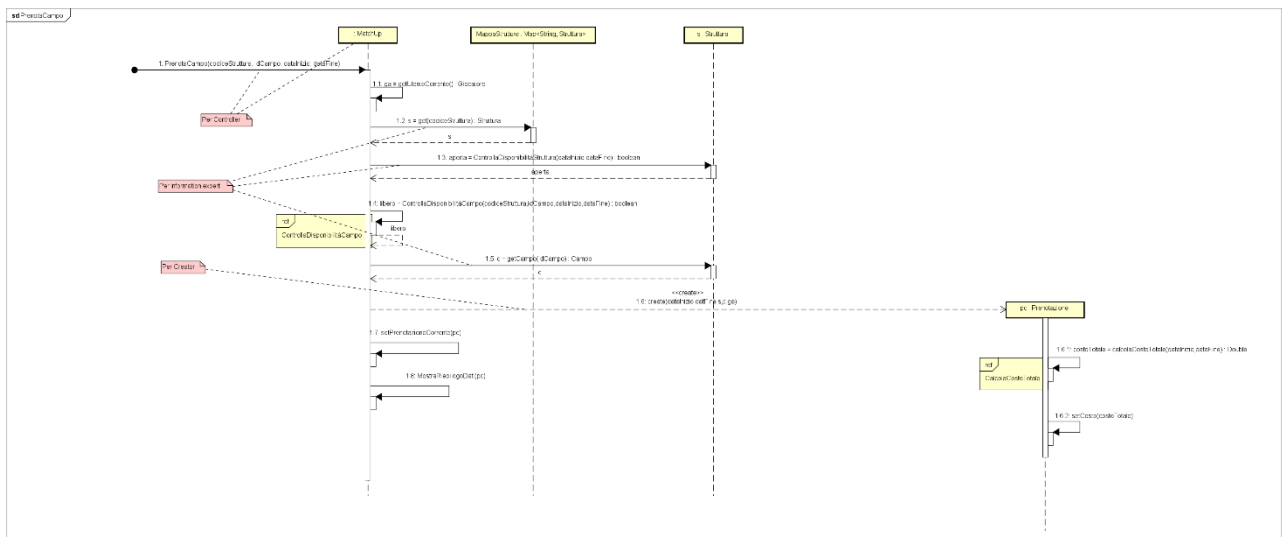


Figura 14: Diagramma di sequenza di Prenota campo

3.3.5 CONTROLLO DISPONIBILITA' CAMPO

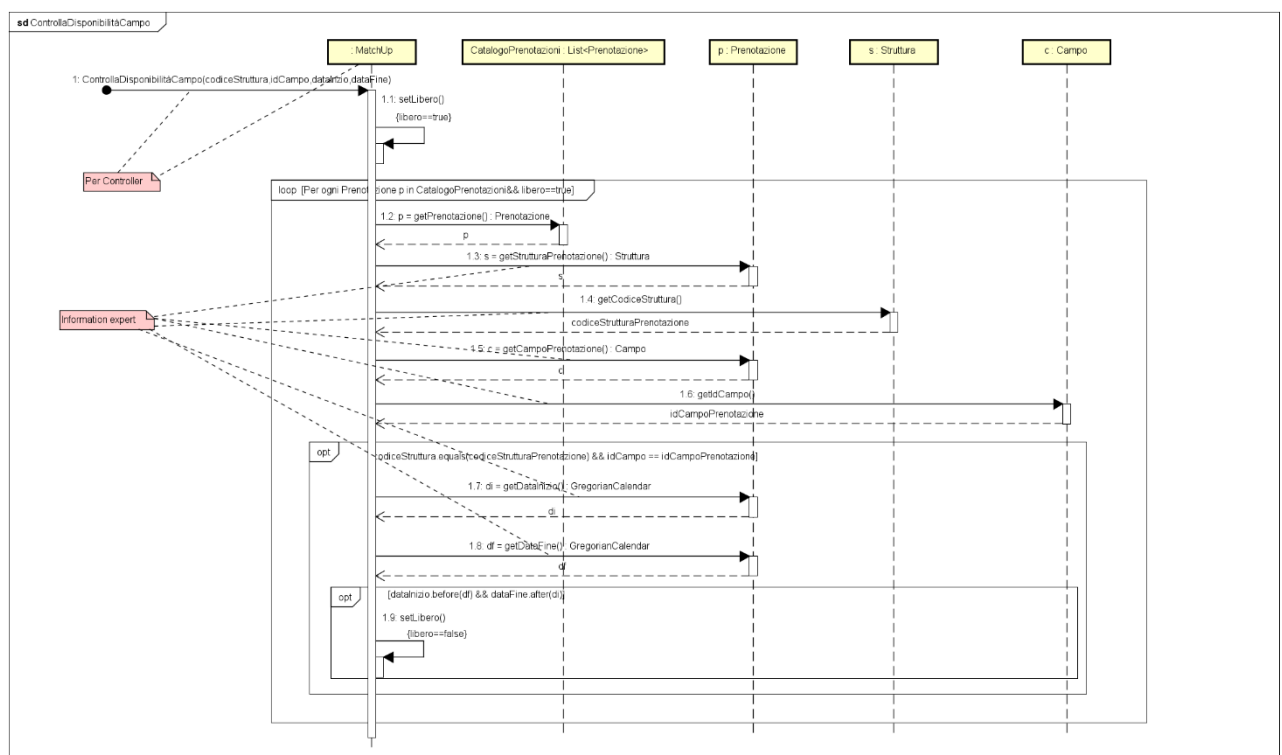


Figura 15: Diagramma di sequenza di Controlla disponibilità campo

3.3.6 CALCOLA COSTO TOTALE

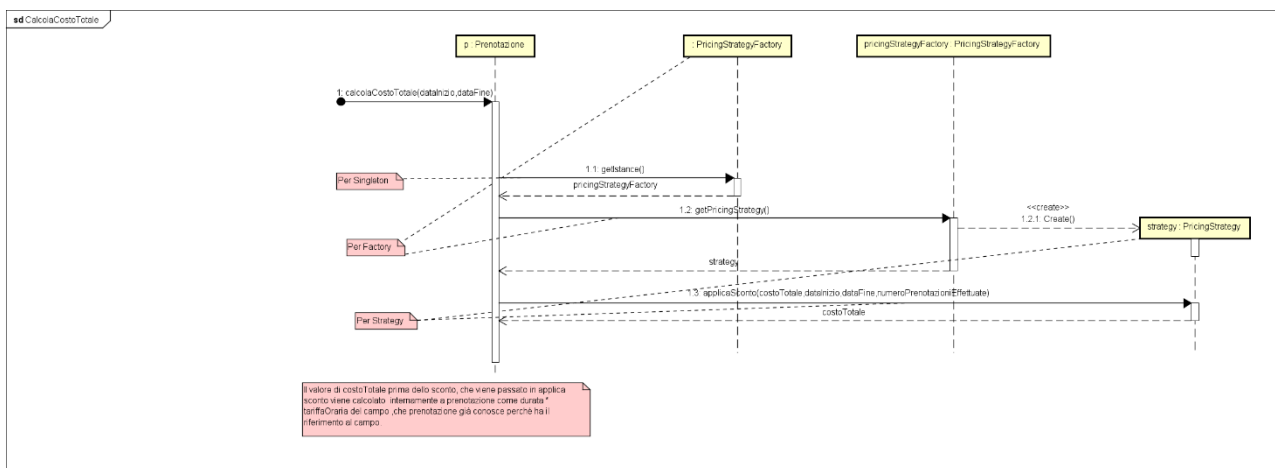


Figura 16: Diagramma di sequenza di Calcola costo totale

3.3.7 CONFERMA PRENOTAZIONE

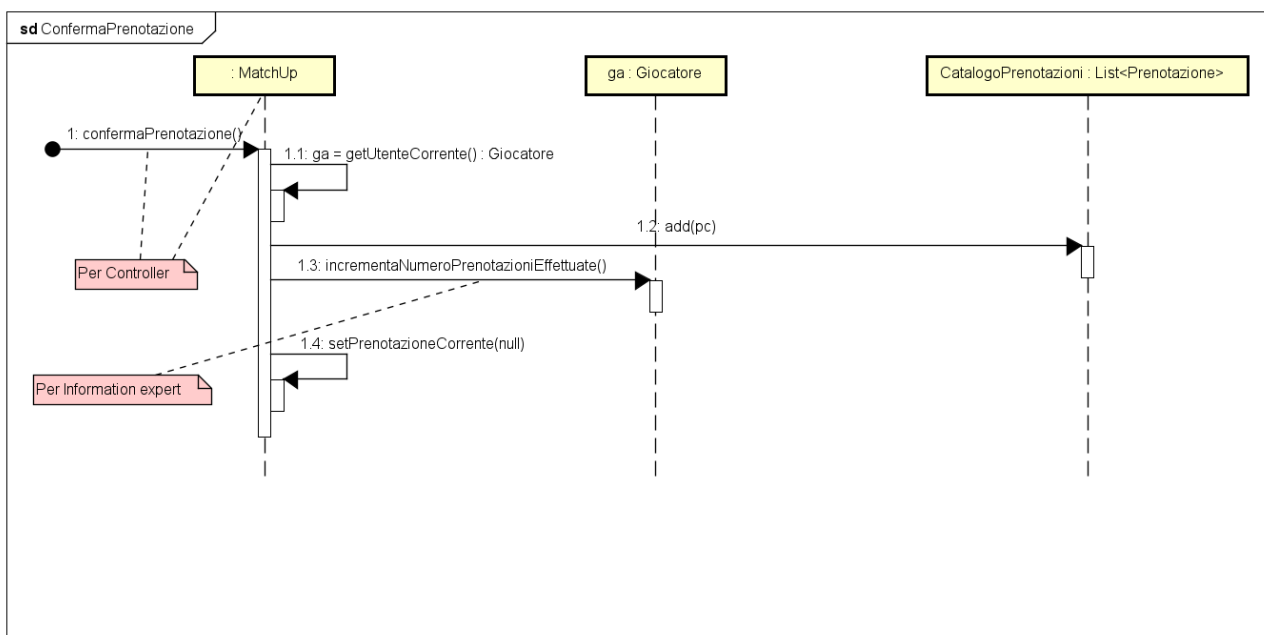
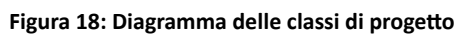


Figura 17: Diagramma di sequenza di Conferma prenotazione

plgProgetti23/24



Nell'iterazione seguente è previsto l'utilizzo dei seguenti pattern GoF , oltre ai comunissimi pattern GRASP usati anche nella prima iterazione (ovvero controller, creator e information expert):

- **Factory:** applicato tramite le classi RegistrationHandlerFactory, RegistrationStrategyFactory e PricingStrategyFactory, ciascuna delegata alla creazione delle istanze concrete rispettivamente di RegistrationHandler, IRegistrationStrategy e IPricingStrategy. In questo modo la responsabilità di decidere quale oggetto concreto istanziare viene centralizzata in una classe dedicata, esterna al client che ne fa uso, riducendo l'accoppiamento tra il codice chiamante e le classi concrete.
- **Singleton:** applicato su MatchUp, in modo da garantire un'unica istanza del controller per l'intera esecuzione del programma, e sulle tre classi factory sopra citate, affinché ciascuna sia istanziata una sola volta e resti accessibile globalmente tramite il proprio metodo getInstance(). Questo pattern è stato quindi abbinato al pattern Factory per ciascuna delle tre factory del sistema.
- **Factory Method:** applicato tramite la classe astratta RegistrationHandler e le sue sottoclassi concrete RegistrationGiocatoreHandler e RegistrationGestoreHandler. Il metodo template istanziaUtente() è definito nella superclasse e delega la creazione dell'istanza concreta al metodo astratto nuovoUtente(), sovrascritto da ciascuna sottoclasse per restituire rispettivamente un Giocatore o un Gestore. In questo modo, il codice chiamante (MatchUp) resta del tutto agnostico rispetto al tipo concreto di utente creato, e l'aggiunta di un'eventuale nuova tipologia di utente in futuro non richiederebbe modifiche al codice esistente, ma la sola introduzione di un nuovo handler.
- **Strategy:** applicato in due punti distinti del sistema.
 - Per il calcolo del costo di una prenotazione, tramite l'interfaccia IPricingStrategy e la relativa implementazione concreta PricingStrategy, ottenuta dalla classe PricingStrategyFactory. Questa scelta consente di modificare o estendere in futuro le politiche di calcolo del prezzo (ad esempio introducendo nuove regole di sconto) definendo una nuova strategia concreta, senza dover intervenire sulla classe Prenotazione.
 - Per la verifica dell'età minima in fase di registrazione, tramite l'interfaccia IRegistrationStrategy e le due implementazioni concrete GiocatoreStrategy e GestoreStrategy, ottenute tramite RegistrationStrategyFactory.

- **Pure Fabrication:** applicato tramite la classe `FasciaOraria`, introdotta per gestire la rappresentazione e l'aggiornamento della disponibilità oraria di un campo in una determinata giornata, senza attribuire questa responsabilità alle classi di dominio `Prenotazione`, `Struttura` o `Campo`, evitando così di aumentarne l'accoppiamento.

5.FASE DI TESTING

5.1 INTRODUZIONE AL TESTING

In questa iterazione gli scenari di testing riguardano le tre funzionalità analizzate: la registrazione di una struttura (UC1), la registrazione di un giocatore (UC2) e il perfezionamento della prenotazione di un campo (UC3), che introduce l'autenticazione e il calcolo del costo basato sullo storico delle prenotazioni effettuate. Come per l'iterazione precedente, la verifica di correttezza dei dati inseriti dall'utente (anagrafici, più username e password) non viene delegata ai singoli metodi di registrazione, bensì viene effettuata a monte, all'interno della classe che gestisce l'interazione con l'utente. Questa scelta implementativa consente di richiedere nuovamente il dato in caso di errore senza interrompere il flusso, e riduce l'accoppiamento evitando di affidare troppi controlli annidati ai metodi che modellano le singole funzionalità.

5.2 SCENARIO REGISTRAZIONE GIOCATORE

In questo scenario un giocatore desidera registrarsi al servizio MatchUp. Lo scenario prevede i seguenti passaggi:

- Inserimento dei dati anagrafici del giocatore: nome, cognome e data di nascita.
- Verifica dell'età del giocatore.
- Conferma da parte del giocatore dei dati inseriti.
- Scelta di uno username.
- Scelta di una password.

Il metodo `istanziaUtente()`, implementato nella classe `MatchUp`, si occupa di creare l'istanza tramite il `RegistrationHandler` appropriato e di verificare, tramite la relativa `IRegistrationStrategy`, che l'età sia superiore al minimo di legge (14 anni per i giocatori, secondo la regola di dominio R6).

I passaggi 4 e 5 richiedono al giocatore di scegliere uno username e una password validi, secondo le seguenti condizioni:

- lo username deve essere univoco nel sistema ;
- la password deve essere lunga almeno 8 caratteri e seguire regole di formattazione.

In definitiva, tramite test di tipo unitario vengono testati i seguenti metodi:

- **istanziaUtente()** ,implementato nella classe MatchUpTest ,verificando i seguenti scenari:

- a) registrazione di un giocatore con età inferiore a 14 anni (scenario di fallimento);
- b) registrazione di un gestore con età inferiore a 18 anni (scenario di fallimento);
- c) registrazione di un giocatore con età superiore a 14 anni (scenario di successo);
- d) registrazione di un gestore con età superiore a 18 anni (scenario di successo).

- **confermaRegistrazione()** / **confermaRegistrazioneGestore()**, verificando i seguenti scenari:

- a) non è presente alcuna registrazione in corso (scenario di fallimento);
- b) è presente una registrazione in corso (scenario di successo), permettendo il salvataggio dell'utente nel sistema.

- **inserisciCredenziali()**, verificando i seguenti scenari:

- a) username già presente nel sistema (scenario di fallimento);
- b) username non presente nel sistema (scenario di successo);
- c) password con lunghezza inferiore a 8 caratteri o con caratteri non validi (scenario di fallimento);
- d) password con lunghezza uguale o superiore a 8 caratteri e che rispetta le regole di formattazione (scenario di successo).

5.3 SCENARIO REGISTRAZIONE STRUTTURA

Lo scenario complementare a quello del giocatore è la registrazione del gestore e della sua struttura.

I passaggi Di inserimento e convalida dei dati anagrafici e non sono identici a quelli descritti nello scenario 5.2, con l'unica differenza che l'entità istanziata è di tipo Gestore anziché Giocatore, e non verranno pertanto ripetuti. Lo scenario prosegue poi con:

- Inserimento dei dati della struttura: nome, località, numero di telefono, orario di apertura e orario di chiusura.
- Inserimento dei dati di uno o più campi della struttura: categoria, tipologia e tariffa oraria. Il passo viene ripetuto finché il gestore lo desidera.

- Conferma dei dati inseriti.

Il metodo **inserisciDatiStruttura()** si occupa di istanziare la struttura e di associarla come struttura corrente; il metodo **confermaRegistrazioneStruttura()** la associa definitivamente a MatchUp e al gestore che l'ha registrata.

Vengono quindi testati:

- **inserisciDatiStruttura()** e **inserisciCampoStruttura()**, verificando la corretta creazione e associazione delle istanze di Struttura e Campo.

- **confermaRegistrazioneStruttura()**, verificando i seguenti scenari:

- a) non è presente alcuna registrazione di struttura in corso (scenario di fallimento);
- b) è presente una registrazione di struttura in corso, associata a un gestore correttamente registrato (scenario di successo).

5.4 SCENARIO PRENOTA CAMPO (UC3 PERFEZIONATO)

Rispetto all'iterazione precedente, lo scenario di prenotazione del campo è stato aggiornato per includere l'autenticazione del giocatore, un controllo aggiuntivo sull'orario di apertura della struttura, la gestione dettagliata delle fasce orarie e il calcolo del costo basato sullo storico delle prenotazioni. Lo scenario di test copre l'intero ciclo di vita della prenotazione, articolato nei seguenti passaggi.

Passaggio 1: Operazione di Login: Prima ancora di tentare una prenotazione, viene testata l'operazione di login() come funzionalità a sé stante all'interno della classe MatchUpTest, essenziale per l'autenticazione di gestori e giocatori .

- **Credenziali errate:** l'inserimento di uno username inesistente o di una password non corrispondente restituisce false e l'attributo utenteCorrente rimane null.
- **Credenziali corrette:** l'inserimento di una coppia username/password valida restituisce true e l'istanza corrispondente viene correttamente assegnata a utenteCorrente.

Passaggio 2: Ricerca, Visualizzazione Strutture e Campi: Avviene tramite i metodi effettuaRicercaStruttura(categoria) e mostraCampiStruttura(codiceStruttura) testati nella classe MatchUpTest.

- **Test su `effettuaRicercaStruttura`:** verifica il filtraggio. Restituisce una lista popolata con le strutture idonee se esistono corrispondenze; restituisce una lista vuota nel caso in cui nessuna struttura contenga campi per la categoria specificata.
- **Test su `mostraCampiStruttura` :** verifica che, fornito il codice di una struttura esistente, il sistema restituisca correttamente la lista delle istanze di Campo ospitate da essa. Fornendo un codice struttura inesistente, il metodo restituisce una lista vuota o solleva un'eccezione gestita.

Passaggio 3: Richiesta Disponibilità e Gestione Fasce Orarie: Questo passaggio testa la visualizzazione degli orari liberi tramite il metodo pubblico `richiediDisponibilità()`, che internamente richiama il metodo privato `mostraFasceOrarie()` per costruire l'oggetto `FasciaOraria` risultante. Quest'ultimo filtra dal catalogo globale le sole prenotazioni relative alla struttura, al campo e al giorno richiesti, aggiornando di conseguenza le fasce orarie occupate tramite il metodo `aggiornaFasciaOraria()`. La verifica avviene per confronto: viene costruito manualmente un oggetto `FasciaOraria` di riferimento, applicando le medesime modifiche corrispondenti alle prenotazioni note per quel campo e quella data, e il suo output viene confrontato con quello restituito dal sistema. Vengono verificati i seguenti scenari:

a) Prenotazioni multiple nello stesso giorno: a fronte di più prenotazioni esistenti per il medesimo campo e la medesima data, le relative fasce orarie risultano correttamente marcate come occupate, mentre le restanti rimangono disponibili. Un ulteriore controllo verifica che, applicando all'oggetto di riferimento una modifica corrispondente a una prenotazione non realmente esistente nel sistema, il confronto con l'output del sistema risulti correttamente negativo.

b) Corretto filtro delle prenotazioni: viene verificato che prenotazioni relative a un giorno o a un campo diverso da quelli richiesti non vengano incluse nel calcolo e non influiscano sul risultato restituito.

Passaggio 4: Generazione e Validazione della Prenotazione (`prenotaCampo`): Il metodo `prenotaCampo()` rappresenta la transizione di stato principale.

a) Verifica Autenticazione: se `utenteCorrente` è null oppure non è un'istanza di `Giocatore` (es. login effettuato da un `Gestore`), il metodo interrompe l'esecuzione e restituisce false.

b) Codice Struttura inesistente: se il codice non trova riscontro in `mappaStrutture`, l'esecuzione si interrompe (false).

c) **Codice Campo inesistente:** se la struttura esiste ma `getCampo(idCampo)` restituisce null, l'esecuzione si interrompe (false).

d) **Struttura non aperta nella fascia oraria richiesta:** viene invocato `controllaDisponibilitàStruttura(dataInizio, dataFine)`. Se l'intervallo richiesto eccede l'orario di apertura/chiusura, il metodo restituisce false.

e) **Fascia oraria del campo già occupata:** tramite `controllaDisponibilitàCampo()` su `catalogoPrenotazioni` vengono verificati i casi di sovrapposizione parziale (destra/sinistra) e contenimento totale. Se c'è sovrapposizione, il metodo restituisce false.

f) **Esito positivo:** se tutti i controlli passano, `MatchUp` istanzia la prenotazione assegnandola a `prenotazioneCorrente`.

Passaggio 5: Verifiche sul Calcolo del Prezzo Il calcolo del prezzo totale viene eseguito mediante il metodo `calcolaCostoTotale()` della classe `Prenotazione`.

- **Nota sulla visibilità:** Poiché `calcolaCostoTotale()` è un metodo private incapsulato all'interno dell'oggetto `Prenotazione`, non è direttamente invocabile tramite test unitari tradizionali dall'esterno della classe.
- **Strategia applicata:** La correttezza della determinazione del costo è stata verificata **indirettamente tramite Test di Integrazione**, sia invocando il flusso pubblico `prenotaCampo()` (confrontando il prezzo finale generato nell'istanza `prenotazioneCorrente`), sia testando direttamente l'output delle singole classi concrete del pattern *Strategy*.

Passaggio 6: Conferma e Persistenza (confermaPrenotazione) Il metodo `confermaPrenotazione()` chiude il flusso.

a) **Esecuzione fuori sequenza:** se `prenotazioneCorrente` è null, restituisce false.

b) **Conferma con dati coerenti:** il metodo restituisce true. La prenotazione viene aggiunta a `catalogoPrenotazioni` (non più al singolo campo) e il numero `PrenotazioniEffettuate` del giocatore viene incrementato. A tal proposito sarà previsto un piccolo test nella classe `GiocatoreTest` che verifica il corretto incremento del numero di prenotazioni.